Another deployment technique is the *canary release*. It's similar to the blue/green deployment, but the traffic from the blue to the green environment is moved gradually over time. The goal is to roll out the change to a small subset of users first, perform some verifications, and then do the same for more and more users until everyone is using the new version.[2] Both blue/green deployments and canary releases provide a straightforward way to roll back changes.

NOTE If you're interested in learning more about deployment and release strategies on Kubernetes, I recommend reading chapter 5 of *Continuous Delivery for Kubernetes* by Mauricio Salatino, published by Manning (https://livebook.manning.com/book/continuous-delivery-for-kubernetes/chapter-5).

Currently, every time you commit changes, a new release candidate is ultimately published and approved if it passes successfully through the commit and acceptance stages. Then you need to copy the version number of the new release candidate and paste it into the Kubernetes manifests before you can update the application in production manually. In the next section, you'll see how to automate that process by implementing the final part of the deployment pipeline: the production stage.

15.3 Deployment pipeline: Production stage

We started implementing a deployment pipeline back in chapter 3, and we have come a long way since then. We've automated all the steps from code commit up to having a release candidate ready for production. There are still two operations that we have performed manually so far: updating the production scripts with the new application version, and deploying it to Kubernetes.

In this section, we'll start looking at the final part of a deployment pipeline, the production stage, and I'll show you how to implement it as a workflow